

```

// Behavioral (RTL) description of a parallel multiplier (n = 8)
module Mult (Product, Multiplicand, Multiplier);
    input [7: 0]    Multiplicand, Multiplier;
    output reg [15: 0] Product;
    always @ (Multiplicand, Multiplier)
        Product = Multiplicand * Multiplier;
endmodule

module Algorithmic_Binary_Multiplier #(parameter dp_width = 5) (
    output [2*dp_width -1: 0] Product, input [dp_width -1: 0] Multiplicand, Multiplier);
    reg [dp_width -1: 0]      A, B, Q; // Sized for datapath
    reg                      C;
    integer                  k;
    assign                   Product = {C, A, Q};
    always @ (Multiplier, Multiplicand) begin
        Q = Multiplier;
        B = Multiplicand;
        C = 0;
        A = 0;
        for (k = 0; k <= dp_width -1; k = k + 1) begin
            if (Q[0]) {C, A} = A + B;
            {C, A, Q} = {C, A, Q} >> 1;
        end
    end
endmodule

module t_Algorithmic_Binary_Multiplier; // Self-checking test bench
    parameter dp_width = 5; // Width of datapath
    wire [2*dp_width -1: 0] Product;
    reg [dp_width -1: 0] Multiplicand, Multiplier;
    integer Exp_Value;
    reg Error;
    Algorithmic_Binary_Multiplier M0 (Product, Multiplicand, Multiplier);
    // Error detection
    initial # 1030000 finish;
    always @ (Product) begin
        Exp_Value = Multiplier * Multiplicand;
        // Exp_Value = Multiplier * Multiplicand +1; // Inject error to confirm detection
        Error = Exp_Value ^ Product;
    end
    // Generate multiplier and multiplicand exhaustively for 5 bit operands
    initial begin
        #5 Multiplicand = 0;
        Multiplier = 0;
        repeat (32) #10 begin Multiplier = Multiplier + 1;
            repeat (32) #5 Multiplicand = Multiplicand + 1;
        end
    end
endmodule

```